

10주차 · 임베딩 서비스 종합 프로젝트와 기술 동향

- 연계 슬라이드: [../slides/week10_capstone_trends.md](#)
- 실습 노트북: [../notebooks/week10.ipynb](#)
- 과제 연계: [../assignments/final_project.md](#)
- 교재 연계: *Hands-On Large Language Models*의 Chapters 2–5, 8–10
- 권장 수업: 설계 50분 · 기술 동향 35분 · 프로젝트 실습 80분 · 상호 검토 35분

1. 좋은 실험을 좋은 데이터 서비스로 바꾸기

최신 임베딩 모델을 호출해 화면에 결과를 보여 주는 것만으로는 종합 프로젝트가 되지 않는다.
좋은 프로젝트는 다음 질문에 증거로 답한다.

- 누가 어떤 결정을 내리기 위해 이 시스템을 사용하는가?
- 단순한 기준선보다 얼마나, 어떤 조건에서 나아졌는가?
- 어디에서 실패하며 그 오류가 누구에게 어떤 피해를 주는가?
- 같은 데이터·설정·커밋으로 결과를 재현할 수 있는가?
- 배포 후 문서·모델·권한이 바뀌면 어떻게 갱신하고 되돌리는가?

전체 증거 사슬은 다음과 같다.

요구사항 → 데이터 → 기준선 → 표현 → 색인·분석 → 평가 → 오류 분석 → 서비스 → 모니터링

어느 한 연결이 끊기면 높은 모델 점수만으로 시스템의 타당성을 방어할 수 없다.

2. 학습목표

1. 데이터-표현-검색/분석-서비스를 전체 시스템으로 설계한다.
2. 기능 요구사항을 측정 가능한 품질·비용·안전 지표로 바꾼다.
3. 기준선, 요소 제거 실험, 오류 분석이 포함된 실험계획을 세운다.

- 4. 모델·색인·말뭉치의 버전을 하나의 배포 단위로 관리한다.
- 5. 2013–2026년 임베딩 기술 흐름을 해결하려는 문제의 변화로 설명한다.
- 6. 최종 보고서·시연·구술평가에서 기술 선택을 근거로 방어한다.

3. 문제 정의 캔버스

“AI로 문서를 검색한다”는 구현 아이디어이지 문제 정의가 아니다. 다음 항목을 한 문장씩 채워 사용자 행동과 성공 조건을 명확히 한다.

항목	답해야 할 질문	예시
사용자	누가 언제 사용하는가?	창업 지원 상담 직원
결정	어떤 판단을 돕는가?	신청 가능 사업 후보 선택
입력	언어·길이·권한은?	한국어 공고문과 사용자 조건
출력	순위·분류·군집·답변 중 무엇인가?	근거가 붙은 사업 목록
정답	누가 관련성을 정의하는가?	상담 지침을 익힌 두 판정자
오류비용	FP와 FN 중 무엇이 더 위험한가?	자격 없는 사업 추천의 행정비용
제약	품질·지연·RAM·비용 한도는?	$R@5 \geq 0.85$, $p95 \leq 1\text{초}$

기능 요구사항 “관련 공고를 찾는다”를 지표로 바꾸면 “고정 최종 평가 질의에서 관련 공고의 $R@5$ 가 0.85 이상”이 된다. 비기능 요구사항 “빠르다”는 “동시 사용자 20명에서 전체 p95 지연시간 1초 이하”처럼 측정 조건을 포함해야 한다.

4. 종단 간 참조 구조

임베딩 서비스의 기본 흐름은 다음과 같다.

출처 버전 관리 → 파싱·체크 → 임베딩·색인 → 검색·재순위화 → API·화면 → 사용자 피드백

이 흐름 전체에 세 가지 횡단 관심사가 적용된다.

- 1. 평가 데이터와 실험 추적: 어떤 변경이 어떤 점수를 만들었는가?
- 2. 접근 제어와 개인정보 보호: 사용자가 볼 수 있는 근거만 검색되는가?

3. 모니터링과 롤백: 새 버전이 실패하면 즉시 이전 버전으로 돌아갈 수 있는가?

노트북은 탐색과 오류 분석에 유용하지만 운영 코드는 재사용 가능한 함수와 설정 파일로 분리한다. 평가표는 사람이 복사하지 말고 실행 결과에서 자동 생성하는 것이 좋다.

5. 기준선 사다리

복잡한 모델은 단순한 방법보다 추가되는 가치가 있을 때만 정당화된다.

- 단계 0: 규칙, 빈도, 무작위 순위
- 단계 1: TF-IDF·BM25와 고전 기계학습
- 단계 2: 기성 임베딩 모델과 정확 탐색
- 단계 3: 희소·밀집 혼합, 재순위화, 미세 조정

예를 들어 BM25의 R@5가 0.78이고 임베딩 검색이 0.81이라면 절대 개선은 0.03, 상대 개선은

$$\frac{0.81 - 0.78}{0.78} \times 100 \approx 3.85\% \quad (1)$$

다. 이 차이에 신뢰구간, 질의별 개선·악화, 지연시간과 비용을 붙여야 한다. BM25가 품번과 수치 질의에서 더 낫다면 혼합 검색의 근거가 된다. 기준선이 복잡한 모델을 이기면 모델을 숨기지 말고 데이터와 과업 정의를 다시 점검한다.

6. 모델 선택표를 작성한다

모델명보다 요구사항과의 대응을 기록한다.

기준	확인할 내용
과업	검색·STS·분류 중 어떤 학습 목표인가?
언어·도메인	한국어와 전문용어를 자체 데이터에서 평가했는가?
입력 길이	실제 p95 길이를 수용하며 잘림률은 얼마인가?
표현	밀집·희소·다중 벡터 중 무엇을 반환하는가?
비용	차원, 매개변수, 처리량, p95 지연시간, RAM은?
운영	라이선스, 오프라인 실행, 모델 버전 고정이 가능한가?

공개 순위표는 후보를 줄이는 자료이지 최종 선택표가 아니다. 같은 모델이라도 질의 접두문, 풀링, 정규화, 출력 차원과 양자화가 바뀌면 검색 순위와 비용이 달라진다.

7. 제품 요구사항에서 평가 데이터를 만든다

평가 데이터는 인터넷에서 편리한 예제를 모은 집합이 아니라 사용자의 입력 분포와 오류 비용을 반영한 측정 도구다.

- 1. 실제 사용자 질문 유형과 빈도를 표본 추출한다.
- 2. 중복, 개인정보와 문서 접근 권한을 처리한다.
- 3. 관련성 판정 기준과 예외를 문서화하고 판정자를 교육한다.
- 4. 다중 정답과 등급형 관련성을 허용한다.
- 5. 언어, 길이, 부정, 수치, 최신성, 표·이미지 여부를 태깅한다.
- 6. 원문 문서나 시간 단위로 학습·개발·최종 평가 데이터를 나눈다.
- 7. 최종 평가 데이터를 잠그고 버전과 해시를 부여한다.

개발 데이터는 모델·청크·임궛값을 선택하는 데 사용하고, 최종 평가 데이터는 선택을 모두 고정한 뒤 성능을 한 번 측정하는 데 사용한다. 공개 벤치마크 재현 결과와 자체 데이터 결과는 둘 다 가치가 있지만 서로 다른 전이 질문에 답한다.

8. 요소 제거 실험으로 기여를 분리한다

요소 제거 실험(ablation study)은 시스템 구성요소 하나를 제거하거나 바꾸고 나머지를 고정해 그 요소의 기여를 추정하는 통제 실험이다.

실험	바꾸는 것	고정할 것
청크	128/256/512토큰	모델·색인·qrels
표현	BM25/밀집/혼합	청크·개발 질의
차원	64/256/전체	모델·프롬프트·색인 계열
재순위화	끔/끔	후보 상위 k 개
프롬프트	지시문 A/B	근거·생성 모델·디코딩

기준 시스템의 nDCG@10이 0.72이고 재순위화를 끄면 0.65라면 관찰된 차이는 0.07이다. 그러나 동시에 후보 수와 청크를 바꾸었다면 이 차이를 재순위화의 기여라고 말할 수 없다. 실험별 설정을 별도의 `run_id` 로 저장하고 한 번에 한 독립변수만 바꾼다.

9. 오류 분석은 다음 실험을 만든다

평균 점수는 무엇을 고쳐야 하는지 말하지 않는다. 개발 데이터의 실패는 다음 순서로 분석해 현재 버전을 개선한다. 최종 평가의 실패도 같은 방식으로 분석하되, 그 결과는 다음 시스템 버전의 개발 자료로 이관하고 기존 최종 평가 점수를 개선 증거로 다시 사용하지 않는다.

실패 수집 → 원인 부호화 → 빈도·피해 계산 → 수정 가설 → 개발 데이터 재평가

권장 오류 분류는 다음과 같다.

- 미등록어·고유명사·품번
- 부정·수치·단위·날짜
- 입력 잘림·긴 문서·다국어
- 불완전한 레이블·qrels, 모호한 질의
- ANN 누락, 잘못된 메타데이터 필터
- 재순위화의 순위 역전
- 근거 누락, 인용 불일치, 근거 없는 생성(환각)

오류표에는 원문 입력, 기대 출력, 실제 출력, 오류 유형, 가능한 원인, 최소 수정 가설을 넣는다. “모델이 나쁨”은 원인 분류가 아니다. “질의의 부정 표현을 무시해 금지 규정을 추천함”처럼 관찰 가능한 현상으로 쓴다. 빈도가 낮아도 피해가 큰 오류는 우선순위가 높을 수 있다.

10. 재현성 패키지

최종 저장소는 다음 역할을 분리하는 것이 좋다.

```

1 | project/
2 |   └─ README.md           # 설치, 실행, 평가 순서
3 |   └─ configs/             # 모델, 체크, 색인, 프롬프트 설정
4 |   └─ data/README.md      # 출처, 스키마, 분할, 라이선스
5 |   └─ src/                 # 재사용 가능한 처리·검색 코드
6 |   └─ notebooks/          # 탐색과 오류 분석
7 |   └─ evaluation/         # qrels, 지표 코드, 고정 실행 결과
8 |   └─ reports/            # 결과표, 데이터·모델 카드

```

최소한 Git 커밋, 패키지 잠금 파일, 난수 시드, 모델 리비전, 하드웨어, 실행시간을 기록한다. 난수 시드만 같다고 완전한 재현성이 보장되지는 않는다. GPU 연산, 라이브러리 버전, 외부 API 모델 변경도 결과를 바꿀 수 있으므로 알려진 비결정성도 문서화한다.

11. 데이터 카드와 모델·시스템 카드

데이터 카드(data card)는 출처, 수집 목적, 동의, 필터, 언어, 결측, 분할, 알려진 편향, 허용·금지 용도를 기록한다. 모델·시스템 카드는 학습 목표, 입력·출력, 모델 리비전, 지표, 부분집합 성능, 비용, 한계, 금지 용도와 모니터링 계획을 기록한다.

카드는 홍보 자료가 아니다. 다음 개발자와 사용자가 “이 데이터를 얼굴 식별에 써도 되는가?”, “이 모델이 숫자 질의에 약한가?”를 판단하게 하는 운영 인터페이스다. 확인하지 못한 학습 데이터 정보는 추측하지 말고 “공개 자료에서 확인 불가”라고 명시한다.

12. 모델과 색인은 한 버전 단위다

임베딩 모델을 바꾸면 벡터 공간이 달라지므로 기존 색인을 그대로 사용할 수 없다. 다음 항목의 조합을 색인 버전으로 관리한다.

```

1 | embedding_model_revision + tokenizer + pooling + normalization
2 | + dimension + chunker_version + corpus_snapshot + index_config
3 | = index_version

```

모델은 같아도 풀링이나 정규화가 다르면 좌표가 바뀐다. 말뭉치 스냅샷이 다르면 문서 ID와 검색 결과도 달라진다. 새 색인을 별도 환경에서 구축해 블루/그린 방식으로 트래픽을 전환하면 문제가 생겼을 때 이전 색인으로 빠르게 롤백할 수 있다.

신규 문서의 증분 추가뿐 아니라 삭제와 권한 변경의 전파를 설계한다. 질의 로그는 개인정보를 최소화하고 품질·지연시간·오류율·응답 보류율을 버전별로 모니터링한다.

13. 성능 예산을 합산한다

전체 p95 지연시간은 각 단계 p95의 단순 합과 정확히 같지는 않지만, 초기 예산을 세울 때는 단계별 상한을 배정할 수 있다.

단계	예산 예시
질의 인코딩	80ms
ANN 후보 검색	40ms
재순위화	250ms
생성	550ms
네트워크·직렬화	80ms
합계	1,000ms

실제 측정에서 재순위화가 430ms라면 전체 모델을 무작정 작게 만들기보다 후보 수, 배치, 교차 인코더를 먼저 점검한다. 처리량은 초당 요청 수, 인코딩은 문서/초, 생성은 토큰/초처럼 단위가 다르므로 단일 숫자로 섞지 않는다.

원시 float32 벡터 저장량은 다음처럼 근사한다.

$$\text{bytes} = N \times d \times 4$$

(2)

문서 200만 개의 768차원 벡터는 약 5.72GiB다. HNSW 그래프, ID와 메타데이터는 별도다. 차원을 줄일 때는 개발 데이터에서 저장량뿐 아니라 품질 회귀를 확인한다. 차원을 고정한 뒤 잠근 최종 평가 데이터에서 한 번 검증한다.

14. 2013–2026년 기술 흐름

연도표는 이전 기술이 사라졌다는 뜻이 아니라 어떤 병목을 새 표현이 해결하려 했는지 보여 준다.

- 2013–2017: Word2Vec, GloVe, fastText가 단어 분포를 정적 벡터로 표현했다.
- 2017–2019: Transformer와 BERT가 문맥별 토큰 표현을, SBERT가 문장 비교 효율을 개선했다.

- 2020–2022: DPR·ColBERT·RAG가 검색과 생성을 연결하고, CLIP이 이미지–텍스트 공간을 정렬했으며, Matryoshka Representation Learning이 가변 차원을 다뤘다.
- 2023–2024: MTEB가 다과업 평가를 정리하고, BGE-M3가 밀집·희소·다중 벡터를 통합하며, ColPali가 시각 문서 검색을 제안했다.
- 2025–2026: Qwen3·Jina·Gemini 계열 공개 연구는 지시문, 다국어, 여러 양식과 유연한 표현을 한 모델에서 다루는 방향을 보였다.

새 방법은 기존 문제를 줄이는 동시에 데이터, 색인, 프롬프트, 평가 범위와 운영 복잡도를 추가한다. BM25와 작은 인코더는 비용·해석·정확 문자열 검색에서 여전히 강한 기준선이다.

15. 최신 동향 1: 과업·표현·양식의 통합

통합은 세 방향으로 진행된다.

- 과업 통합: 임베딩과 재순위화를 같은 기반 모델·지시문 체계에서 제공한다.
- 표현 통합: 밀집·희소·다중 벡터를 하나의 모델에서 생성한다.
- 양식 통합: 텍스트·이미지·오디오·비디오를 공동 공간에 배치한다.

운영 모델 수가 줄 수 있지만 과업별 지시문, 어댑터, 출력 형식과 색인은 여전히 검증해야 한다. 통합 모델 하나를 선택했다고 평가가 하나로 통합되는 것은 아니다.

16. 논문 도판 읽기: 임베딩과 재순위화

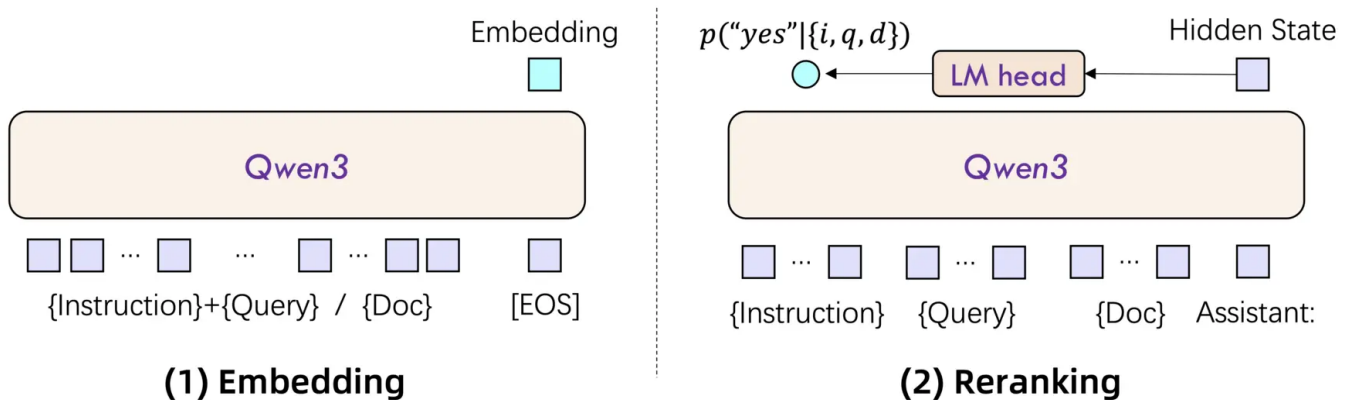


그림 1. Qwen3-Embedding은 입력별 독립 벡터를 만들고 Qwen3-Reranker는 질의와 문서를 함께 읽어 관련성 점수를 계산한다. 출처: Zhang et al. (2025), Figure 1, [원 논문](#).

도판에서 두 경로를 구분해 읽는다. 임베딩 경로는 질의와 문서를 독립적으로 처리하므로 문서 벡터를 미리 색인할 수 있다. 재순위화 경로는 질의-문서 쌍을 함께 입력해 더 풍부한 상호작용을 계산하지만 모든 말뭉치에 적용하기에는 비싸다. 따라서 넓은 후보 생성과 좁은 후보의 정밀 재순위화라는 역할 분담이 자연스럽다.

두 모델을 함께 썼다고 하나의 점수만 보고해서는 안 된다. 후보 단계의 R@100과 재순위화 후 nDCG@10을 분리해야 어느 단계가 개선되었는지 알 수 있다. 해당 논문은 2025년 공개 연구이므로 사용 시 모델 리비전과 평가 조건을 함께 확인한다.

17. 최신 동향 2: 효율과 선택 가능한 표현

- Matryoshka 표현: 긴 벡터의 앞부분도 유용하도록 학습해 차원별 품질-비용 선택을 돕는다.
- 양자화: `int8` , 이진 코드, 곱 양자화로 저장·전송·거리 계산을 줄인다.
- 단계적 검색: 단일 벡터로 넓게 찾고 다중 벡터로 정밀하게 비교한다.
- 지식 증류: 큰 모델의 학습 신호로 작은 인코더를 훈련한다.
- 합성 데이터: 질의와 어려운 비관련 예시를 생성해 도메인 학습을 보완한다.
- 장문·적응형 체크: 문서 구조와 질의에 따라 표현 단위를 조정한다.

차원을 1024에서 256으로 줄이면 원시 벡터 저장량은 1/4이지만 품질이 정확히 유지된다고 가정할 수 없다. 개발 qrels로 R@k와 nDCG를 다시 측정하고 부분집합별 손실을 확인한다. 차원을 고정한 뒤 잠근 최종 평가 데이터에서 한 번 검증한다.

18. 최신 동향 3: 평가 범위의 확대

MTEB에서 MMTEB로 이어지는 흐름은 순위표의 모델 수보다 평가 범위를 넓히는 데 의미가 있다. 더 많은 언어·문자 체계, 검색 이외의 과업, 지시문 처리, 모델 크기·공개성·비용, 평가 데이터 오염을 함께 본다. 하나의 전체 평균은 여전히 강점과 약점을 숨긴다.

2026년 사전 공개 논문의 수치와 순위는 후속 버전에서 바뀔 수 있다. 보고서에는 논문·모델의 확인 날짜와 버전을 적고, 자체 고정 최종 평가 데이터 결과를 우선한다. “최신”은 선택 근거가 아니라 재검증이 필요한 상태 정보다.

19. 윤리·법·안전 게이트

배포 전에 다음 질문에 예/아니오와 증거로 답한다.

- 데이터 수집·재사용·모델 라이선스가 허용되는가?
- 개인정보와 민감 정보가 임베딩·로그·캐시에 남는가?
- 사용자 권한이 검색 전에 강제되는가?
- 언어·집단별 오류와 가장 큰 피해를 측정했는가?
- 근거, 자동화 범위와 모델 한계를 화면에서 알리는가?
- 삭제·정정·이의제기·롤백 경로가 실제로 작동하는가?

고위험 용도에서는 평균 정확도보다 최악의 실패와 사람 검토 체계가 더 중요할 수 있다.
법적 적합성은 기술 지표만으로 판단하지 말고 담당 조직의 검토를 거친다.

20. Hands-On LLM 전체 연결

프로젝트 단계	교재	종합 질문
표현 이해	Chapters 2–3	토큰·은닉 상태·풀링이 무엇을 보존하는가?
분류·군집	Chapters 4–5	시각화와 지표가 실제 구조를 지지하는가?
검색·RAG	Chapter 8	청크·색인·재순위화 중 병목은 어디인가?
멀티모달	Chapter 9	OCR 텍스트와 시각 레이아웃 중 무엇을 보존할까?
임베딩 학습	Chapter 10	데이터·손실함수·평가기가 같은 관련성을 정의하는가?

교재 코드를 그대로 실행한 화면은 프로젝트 결과가 아니다. 자체 데이터, 기준선, 지표, 실패 사례를 추가해 코드가 어떤 주장을 검증하는지 보여 주어야 한다.

21. 노트북 실습 안내와 점검 지점

[week10.ipynb](#) 는 종합 프로젝트의 증거를 표 형태로 채우는 골격이다.

점검 1 · 문제정의 사전 작성

코드를 실행하기 전에 `project` 의 사용자, 의사결정, 데이터 출처, 기준선, 주요 지표와 가장 위험한 오류를 완전한 문장으로 작성한다. `Recall@5` 등 처럼 선택을 미루는 표현을 실제 지표와 최소 성공 기준으로 바꾼다.

점검 2 · 실험표 설계

`experiments` 에는 최소 세 행을 만든다: 기준선, 임베딩 방법, 개발 개선 실험. 모든 행에서 같은 데이터 분할과 평가 집합을 사용한다. `run_id` 가 설정 파일이나 코드 커밋으로 추적되는지, 지연시간과 메모리의 측정 조건이 같은지 확인한다.

점검 3 · 오류 분석표

최종 평가의 서로 다른 실패 사례를 최소 5개 기록한다. `possible_cause` 는 관찰한 증거와 추론을 구분하고 `proposed_fix` 에는 다시 검증할 지표를 쓴다. 모든 개선안을 구현할 필요는 없지만 가장 중요한 하나는 개발 데이터에서 검증한다.

점검 4 · 모델 카드

모델 이름·버전·사용법, 권장·금지 범위, 최종 평가 데이터와 지표, 대표 실패, 실행 환경을 작성한다. 확인 가능한 범위에서 학습 데이터, 개인정보·저작권·편향, 계산 자원과 비용을 덧붙인다. 모르는 내용을 생성형 AI로 추측해 채우지 않는다.

점검 5 · 발표 리허설

10분 발표를 문제 1분, 데이터·기준선 2분, 임베딩 방법 2분, 평가와 개선 실험 2분, 오류·한계 1분, 새 입력 시연과 다음 단계 2분으로 구성한다. 모델 이름보다 선택 근거와 실패 분석에 더 많은 시간을 배정한다.

22. 최종 산출물 체크리스트

1. 사용자와 측정 가능한 성공 기준이 있는 문제정의서
2. 출처·라이선스·분할·편향을 기록한 데이터 카드
3. 고정 개발·최종 평가 데이터와 `qrels`
4. 빈 환경에서 실행 가능한 코드·노트북·설정

5. 기준선 1개와 임베딩 방법 1개의 공정한 비교
6. 개발 데이터에서 수행한 개선 실험 1개
7. 서로 다른 실패 사례 5개와 원인·피해 분석
8. 최종 품질 지표와 질의별 결과
9. 안전·개인정보·권한·라이선스 검토
10. 10분 발표·새 입력 시연·5분 질의응답

제출 전 새 환경에서 README 순서대로 실행한다. 결과표가 자동 생성되는지, 최종 평가 데이터로 설정을 바꾸지 않았는지, 데이터 경로와 모델 리비전이 기록되었는지 확인한다.

23. 자주 하는 오해

- **오해:** 최신·대형 모델을 사용하면 프로젝트의 독창성이 높다.

교정: 자체 문제에서 기준선 대비 개선과 실패 원인을 증명하는 것이 핵심이다.

- **오해:** 요소 제거 실험은 여러 설정을 한꺼번에 바꾸는 비교다.

교정: 한 요소의 기여를 보려면 나머지 조건을 고정해야 한다.

- **오해:** 노트북이 실행되면 재현 가능한 시스템이다.

교정: 데이터·환경·모델 버전·실행 순서와 비결정성까지 기록해야 한다.

- **오해:** 모델을 바꾸어도 차원이 같으면 기존 색인을 쓸 수 있다.

교정: 차원이 같아도 좌표 공간이 다르므로 다시 임베딩하고 색인을 구축해야 한다.

- **오해:** 평균 점수가 좋아지면 배포해도 된다.

교정: 위험한 오류, 부분집합, 지연시간, 권한과 롤백까지 통과해야 한다.

- **오해:** 통합 모델은 모든 과업을 한 번의 평가로 검증할 수 있다.

교정: 양식·과업·표현 방식별 입력과 실패가 달라 별도 평가가 필요하다.

24. 핵심 정리

1. 좋은 프로젝트는 기술 이름보다 의사결정의 증거 사슬이 명확하다.
2. 기준선과 요소 제거 실험이 각 구성요소의 추가 가치를 보여 준다.

3. 오류 분석은 평균 점수를 다음 검증 가능한 개선 가설로 바꾼다.
4. 모델·체크·말뭉치·색인 설정을 하나의 버전 단위로 관리한다.
5. 품질·비용·안전·재현성을 함께 만족해야 데이터 서비스가 된다.
6. 최신 기술은 자체 데이터와 운영 제약에서 다시 검증해야 한다.

25. 복습·구술 문제

1. “AI 문서 검색”을 사용자·결정·입력·출력·오류비용이 있는 문제로 다시 정의하라.
2. 기준선이 복잡한 모델을 이겼을 때 확인할 데이터·설정 문제를 세 가지 제시하라.
3. 체크 크기 요소 제거 실험에서 바꾸는 것과 고정할 것을 설명하라.
4. 최종 평가 누수를 막기 위한 데이터 분할과 실험 절차를 설명하라.
5. 모델 리비전은 같지만 풀링이 바뀌었을 때 색인을 다시 만들어야 하는 이유는 무엇인가?
6. 전체 p95 지연시간이 초과됐을 때 병목을 찾는 측정 계획을 세워라.
7. Qwen3 도판에서 임베딩과 재순위화의 계산·색인 차이를 설명하라.
8. 예산이 절반이면 유지할 품질·안전 조건과 줄일 구성요소를 선택하고 근거를 제시하라.
9. 새 문서 추가, 문서 삭제, 사용자 권한 변경을 색인에 반영하는 절차를 설명하라.
10. 프로젝트에서 가장 피해가 큰 실패 하나와 이를 감시할 배포 지표를 제시하라.

26. 참고문헌

1. Mikolov, T. et al. (2013). “Efficient Estimation of Word Representations in Vector Space.” [arXiv](#)
2. Vaswani, A. et al. (2017). “Attention Is All You Need.” [NeurIPS](#)
3. Reimers, N., & Gurevych, I. (2019). “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” [EMNLP](#)
4. Lewis, P. et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” [arXiv](#)
5. Kusupati, A. et al. (2022). “Matryoshka Representation Learning.” [NeurIPS](#)
6. Muennighoff, N. et al. (2023). “MTEB: Massive Text Embedding Benchmark.” [EACL](#)

7. Zhang, Y. et al. (2025). "Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models." [arXiv](#)
8. Alammar, J., & Grootendorst, M. (2024). *Hands-On Large Language Models*. [공개 코드](#)
9. Li, M. et al. (2026). "Qwen3-VL-Embedding and Qwen3-VL-Reranker: A Unified Framework for State-of-the-Art Multimodal Retrieval and Ranking." 사전 공개 논문. [arXiv](#)
10. Shanbhogue, M. et al. (2026). "Gemini Embedding 2: A Native Multimodal Embedding Model from Gemini." 사전 공개 논문. [arXiv](#)
11. Hönicke, F. et al. (2026). "jina-embeddings-v5-omni: Geometry-preserving Embeddings via Locked Aligned Towers." 사전 공개 논문. [arXiv](#)